

TestResultDBAccess

v. 0.1.5

Tran Duy Ngoan

10.04.2025

Contents

1	Introduction	1
2	Description	2
2.0.1	DBAccessFactory	2
2.0.2	DBAccess Interface	2
2.0.3	DirectDBAccess	2
2.0.4	RestApiDBAccess	2
3	db_access_interface.py	3
3.1	Class: DBAccessInterface	3
3.1.1	Method: connect	3
3.1.2	Method: disconnect	3
3.1.3	Method: commit	3
3.1.4	Method: arGetCategories	3
3.1.5	Method: bExistingResultID	3
3.1.6	Method: sGetLatestFileID	3
3.1.7	Method: sCreateNewTestResult	3
3.1.8	Method: nCreateNewFile	3
3.1.9	Method: vCreateNewHeader	4
3.1.10	Method: nCreateNewSingleTestCase	4
3.1.11	Method: nCreateNewTestCase	4
3.1.12	Method: vCreateAbortReason	4
3.1.13	Method: vCreateCCRdata	4
3.1.14	Method: vCreateTags	4
3.1.15	Method: vCreateReanimation	4
3.1.16	Method: vSetCategory	4
3.1.17	Method: vUpdateFileEndTime	4
3.1.18	Method: vUpdateResultEndTime	4
3.1.19	Method: vUpdateEvtbl	4
3.1.20	Method: vUpdateEvtbls	4
3.1.21	Method: vFinishTestResult	4
4	direct_db_access.py	5
4.1	Class: DirectDBAccess	5
4.1.1	Method: connect	5
4.1.2	Method: commit	6
4.1.3	Method: disconnect	6
4.1.4	Method: cleanAllTables	6

4.1.5	Method: sCreateNewTestResult	6
4.1.6	Method: nCreateNewFile	7
4.1.7	Method: vCreateNewHeader	8
4.1.8	Method: nCreateNewSingleTestCase	10
4.1.9	Method: nCreateNewTestCase	11
4.1.10	Method: vCreateTags	13
4.1.11	Method: vSetCategory	13
4.1.12	Method: vUpdateStartEndTime	13
4.1.13	Method: arGetCategories	14
4.1.14	Method: vCreateAbortReason	14
4.1.15	Method: vCreateReanimation	14
4.1.16	Method: vCreateCCRdata	14
4.1.17	Method: vFinishTestResult	15
4.1.18	Method: vUpdateEvtbls	15
4.1.19	Method: vUpdateEvtbl	15
4.1.20	Method: vEnableForeignKeyCheck	15
4.1.21	Method: sGetLatestFileID	16
4.1.22	Method: vUpdateFileEndTime	16
4.1.23	Method: vUpdateResultEndTime	16
4.1.24	Method: bExistingResultID	16
4.1.25	Method: arGetProjectVersionSWByID	17
5	rest_api_db_access.py	18
5.1	Class: RestApiDBAccess	18
5.1.1	Method: get_certs_file	18
5.1.2	Method: encrypt_password	18
5.1.3	Method: connect	18
5.1.4	Method: disconnect	19
5.1.5	Method: arGetCategories	19
5.1.6	Method: bExistingResultID	19
5.1.7	Method: sGetLatestFileID	20
5.1.8	Method: arGetProjectVersionSWByID	20
5.1.9	Method: sCreateNewTestResult	20
5.1.10	Method: nCreateNewFile	21
5.1.11	Method: vCreateNewHeader	22
5.1.12	Method: nCreateNewSingleTestCase	24
5.1.13	Method: nCreateNewTestCase	25
5.1.14	Method: vCreateAbortReason	25
5.1.15	Method: vCreateCCRdata	26
5.1.16	Method: vCreateTags	26
5.1.17	Method: vCreateReanimation	26
5.1.18	Method: vSetCategory	26
5.1.19	Method: vUpdateFileEndTime	27
5.1.20	Method: vUpdateResultEndTime	27
5.1.21	Method: vFinishTestResult	27
5.1.22	Method: vUpdateEvtbl	28

5.1.23 Method: vUpdateEvtbls	28
6 DBAccessFactory.py	29
6.1 Class: DBAccessFactory	29
6.1.1 Method: create	29
7 Appendix	30
8 History	31

Chapter 1

Introduction

The **TestResultDBAccess** package is designed to streamline the process of importing test data into the database of a test result web application.

This package offers two distinct access methods: **Direct Access** and **REST API Access**.

Users can seamlessly choose between these methods based on their preferences and requirements.

- **Direct Access:** This method provides a direct connection to the database using the SQL protocol, allowing efficient data retrieval and import operations.
- **REST API Access:** This method interacts with the web application via HTTP/HTTPS protocol, providing flexibility and compatibility with modern web architectures.

The package's DB Access Factory facilitates the creation of instances for both **Direct Access** and **REST API Access**, ensuring a smooth and adaptable workflow for importing test data into the target database.

Whether users prefer direct database interaction or web-based API access, **TestResultDBAccess** offers a comprehensive solution to meet diverse testing needs.

Chapter 2

Description

The `TestResultDBAccess` package offers a flexible and efficient way to interact with the database of a test result web application.

Users can choose between direct database access and REST API access, depending on their specific needs and preferences.

2.0.1 DBAccessFactory

The **DBAccessFactory** class is responsible for creating instances of the **DBAccess** interface based on the specified access method.

It provides the following method:

- **create(access_method: str): DBAccess** - Creates an instance of **DBAccess** based on the specified access method (`rest` or `db`).

2.0.2 DBAccess Interface

The **DBAccess** interface defines a set of methods for interacting with the database. These methods include:

- **connect(host: str, user: str, passwd: str, database: str)** - Establishes a connection to the database.
- **disconnect()** - Destroys the connection to the database.
- Various methods for retrieving, creating, updating, and finishing test result data in the database, such as **arGetCategories()**, **sGetLatestFileID()**, **vCreateTags()**, **vUpdateResultEndTime()**, and **vFinishTestResult()**.

2.0.3 DirectDBAccess

The **DirectDBAccess** class implements the **DBAccess** interface for direct database access. It provides methods for establishing and destroying connections, as well as implementing the various data manipulation methods defined in **DBAccess**.

2.0.4 RestApiDBAccess

The **RestApiDBAccess** class implements the **DBAccess** interface for REST API access to the database. Similar to **DirectDBAccess**, it provides methods for connecting, disconnecting, and implementing data manipulation methods through the REST API.

Chapter 3

db_access_interface.py

3.1 Class: DBAccessInterface

Imported by:

```
from TestResultDBAccess.DBAccess.db_access_interface import DBAccessInterface
```

Abstract base class defining the interface for database access.

This interface defines methods for connecting to and disconnecting from the database, as well as methods for retrieving, creating, updating, and calling stored procedures in the database.

3.1.1 Method: connect

Connects to the database using the provided connection parameters.

3.1.2 Method: disconnect

Disconnects from the database.

3.1.3 Method: commit

Commits a transaction (only applicable for DirectDBAccess with transaction support).

3.1.4 Method: arGetCategories

Retrieves categories from the database.

3.1.5 Method: bExistingResultID

Checks if the given result ID exists in the database.

3.1.6 Method: sGetLatestFileID

Retrieves the latest file ID from the database.

3.1.7 Method: sCreateNewTestResult

Creates a new test result record in the database.

3.1.8 Method: nCreateNewFile

Creates a new file record in the database.

3.1.9 Method: vCreateNewHeader

Creates a new file header record in the database.

3.1.10 Method: nCreateNewSingleTestCase

Creates a new single test case record in the database.

3.1.11 Method: nCreateNewTestCase

Creates new test case(s) record in the database.

3.1.12 Method: vCreateAbortReason

Creates a new abort reason record in the database.

3.1.13 Method: vCreateCCRdata

Creates new CCR data in the database.

3.1.14 Method: vCreateTags

Creates new tags in the database.

3.1.15 Method: vCreateReanimation

Updates an existing test result with reanimation data in the database.

3.1.16 Method: vSetCategory

Updates an existing test result with category information in the database.

3.1.17 Method: vUpdateFileEndTime

Updates an existing file record with end time in the database.

3.1.18 Method: vUpdateResultEndTime

Updates an existing test result record with end time in the database.

3.1.19 Method: vUpdateEvtbl

Calls a stored procedure to update event tables in the database.

3.1.20 Method: vUpdateEvtbls

Calls a stored procedure to update event tables (plural) in the database.

3.1.21 Method: vFinishTestResult

Calls a stored procedure to finish a test result in the database.

Chapter 4

direct_db_access.py

4.1 Class: DirectDBAccess

Imported by:

```
from TestResultDBAccess.DBAccess.direct_db_access import DirectDBAccess
```

DirectDBAccess class play a role as mysqlclient and provide methods to interact with TestResultWebApp's database.

4.1.1 Method: connect

Connect to the database with provided authentication and db info.

Arguments:

- `host`
/ *Condition*: required / *Type*: str /
URL which is hosted the TestResultWebApp's database.
- `user`
/ *Condition*: required / *Type*: str /
User name for database authentication.
- `passwd`
/ *Condition*: required / *Type*: str /
User's password for database authentication.
- `database`
/ *Condition*: required / *Type*: str /
Database name.
- `charset`
/ *Condition*: optional / *Type*: str / *Default*: 'utf8' /
The connection character set.
- `use_unicode`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, CHAR and VARCHAR and TEXT columns are returned as Unicode strings, using the configured character set.

Returns:

(no returns)

4.1.2 Method: commit

Commit changes within transaction.

Arguments:

(no arguments)

Returns:

(no returns)

4.1.3 Method: disconnect

Disconnect from TestResultWebApp's database.

Arguments:

(no arguments)

Returns:

(no returns)

4.1.4 Method: cleanAllTables

Delete all table data. Please be careful before calling this method.

Arguments:

(no arguments)

Returns:

(no returns)

4.1.5 Method: sCreateNewTestResult

Creates a new test result in `tbl_result`. This is the main table which is linked to all other data by means of `test_result_id`.

Arguments:

- `tbl_prj_project`
/ Condition: required / Type: str /
Project information.
- `tbl_prj_variant`
/ Condition: required / Type: str /
Variant information.
- `tbl_prj_branch`
/ Condition: required / Type: str /
Branch information.
- `tbl_test_result_id`
/ Condition: required / Type: str /
UUID of test result.
- `tbl_result_interpretation`
/ Condition: required / Type: str /
Result interpretation.
- `tbl_result_time_start`
/ Condition: required / Type: str /
Test result start time as format `%Y-%m-%d %H:%M:%S`.

- `.tbl_result_time_end`
/ *Condition*: required / *Type*: str /
Test result end time as format %Y-%m-%d %H:%M:%S.
- `.tbl_result_version_sw_target`
/ *Condition*: required / *Type*: str /
Software version information.
- `.tbl_result_version_sw_test`
/ *Condition*: required / *Type*: str /
Test version information.
- `.tbl_result_version_target`
/ *Condition*: required / *Type*: str /
Hardware version information.
- `.tbl_result_jenkinsurl`
/ *Condition*: required / *Type*: str /
Jenkins's url in case test result is executed by jenkins.
- `.tbl_result_reporting_qualitygate`
/ *Condition*: required / *Type*: str /
Quality gate information for reporting.

Returns:

- `.tbl_test_result_id`
/ *Type*: str /
`test_result_id` of new test result.

4.1.6 Method: nCreateNewFile

Create new file entry in `tbl_file` table.

Arguments:

- `.tbl_file_name`
/ *Condition*: required / *Type*: str /
File name information.
- `.tbl_file_tester_account`
/ *Condition*: required / *Type*: str /
Tester account information.
- `.tbl_file_tester_machine`
/ *Condition*: required / *Type*: str /
Test machine information.
- `.tbl_file_time_start`
/ *Condition*: required / *Type*: str /
Test file start time as format %Y-%m-%d %H:%M:%S.
- `.tbl_file_time_end`
/ *Condition*: required / *Type*: str /
Test file end time as format %Y-%m-%d %H:%M:%S.

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result for linking to `tbl_result` table.
- `_tbl_file_origin`
/ *Condition*: required / *Type*: str /
Origin (test framework) of test file. Default is "ROBFW"

Returns:

- `iInsertedID`
/ *Type*: int /
ID of new entry.

4.1.7 Method: vCreateNewHeader

Create a new header entry in `tbl_file.header` table which is linked with the file.

Arguments:

- `_tbl_file_id`
/ *Condition*: required / *Type*: int /
File ID information.
- `_tbl_header_testtoolconfiguration_testtoolname`
/ *Condition*: required / *Type*: str /
Test tool name.
- `_tbl_header_testtoolconfiguration_testtoolversionstring`
/ *Condition*: required / *Type*: str /
Test tool version.
- `_tbl_header_testtoolconfiguration_projectname`
/ *Condition*: required / *Type*: str /
Project name.
- `_tbl_header_testtoolconfiguration_logfileencoding`
/ *Condition*: required / *Type*: str /
Encoding of logfile.
- `_tbl_header_testtoolconfiguration_pythonversion`
/ *Condition*: required / *Type*: str /
Python version info.
- `_tbl_header_testtoolconfiguration_testfile`
/ *Condition*: required / *Type*: str /
Test file name.
- `_tbl_header_testtoolconfiguration_logfilepath`
/ *Condition*: required / *Type*: str /
Path to log file.
- `_tbl_header_testtoolconfiguration_logfilemode`
/ *Condition*: required / *Type*: str /
Mode of log file.

- `.tbl.header.testtoolconfiguration.ctrlfilepath`
/ *Condition*: required / *Type*: str /
Path to control file.
- `.tbl.header.testtoolconfiguration.configfile`
/ *Condition*: required / *Type*: str /
Path to configuration file.
- `.tbl.header.testtoolconfiguration.confname`
/ *Condition*: required / *Type*: str /
Configuration name.
- `.tbl.header.testfileheader.author`
/ *Condition*: required / *Type*: str /
File author.
- `.tbl.header.testfileheader.project`
/ *Condition*: required / *Type*: str /
Project information.
- `.tbl.header.testfileheader.testfiledate`
/ *Condition*: required / *Type*: str /
File creation date.
- `.tbl.header.testfileheader.version.major`
/ *Condition*: required / *Type*: str /
File major version.
- `.tbl.header.testfileheader.version.minor`
/ *Condition*: required / *Type*: str /
File minor version.
- `.tbl.header.testfileheader.version.patch`
/ *Condition*: required / *Type*: str /
File patch version.
- `.tbl.header.testfileheader.keyword`
/ *Condition*: required / *Type*: str /
File keyword.
- `.tbl.header.testfileheader.shortdescription`
/ *Condition*: required / *Type*: str /
File short description.
- `.tbl.header.testexecution.useraccount`
/ *Condition*: required / *Type*: str /
Tester account who run the execution.
- `.tbl.header.testexecution.computername`
/ *Condition*: required / *Type*: str /
Machine name which is executed on.
- `.tbl.header.testrequirements.documentmanagement`
/ *Condition*: required / *Type*: str /
Requirement management information.

- `tbl_header.testrequirements_testenvironment`
/ *Condition*: required / *Type*: str /
Requirement environment information.
- `tbl_header.testbenchconfig_name`
/ *Condition*: required / *Type*: str /
Test bench configuration name.
- `tbl_header.testbenchconfig_data`
/ *Condition*: required / *Type*: str /
Test bench configuration data.
- `tbl_header.preprocessor_filter`
/ *Condition*: required / *Type*: str /
Preprocessor filter information.
- `tbl_header.preprocessor_parameters`
/ *Condition*: required / *Type*: str /
Preprocessor parameters definition.

Returns:*(no returns)***4.1.8 Method: nCreateNewSingleTestCase**Create single testcase entry in `tbl_case` table immediately.**Arguments:**

- `tbl_case_name`
/ *Condition*: required / *Type*: str /
Test case name.
- `tbl_case_issue`
/ *Condition*: required / *Type*: str /
Test case issue ID.
- `tbl_case_tcid`
/ *Condition*: required / *Type*: str /
Test case ID (used for testmanagement tool).
- `tbl_case_fid`
/ *Condition*: required / *Type*: str /
Test case requirement (function) ID.
- `tbl_case_testnumber`
/ *Condition*: required / *Type*: int /
Order of test case in file.
- `tbl_case_repeatcount`
/ *Condition*: required / *Type*: int /
Test case repeatition count.
- `tbl_case_component`
/ *Condition*: required / *Type*: str /
Component which test case is belong to.

- `tbl.case.time_start`
/ *Condition*: required / *Type*: str /
Test case start time as format %Y-%m-%d %H:%M:%S.
- `tbl.case.result_main`
/ *Condition*: required / *Type*: str /
Test case main result.
- `tbl.case.result_state`
/ *Condition*: required / *Type*: str /
Test case completion state.
- `tbl.case.result_return`
/ *Condition*: required / *Type*: int /
Test case result code (as integer).
- `tbl.case.counter_resets`
/ *Condition*: required / *Type*: int /
Counter of target reset within test case execution.
- `tbl.case.lastlog`
/ *Condition*: required / *Type*: str /
Traceback information when test case is failed.
- `tbl.test.result_id`
/ *Condition*: required / *Type*: str /
UUID of test result for linking to file in `tbl.result` table.
- `tbl.file.id`
/ *Condition*: required / *Type*: int /
Test file ID for linking to file in `tbl.file` table.

Returns:

- `iInsertedID`
/ *Type*: int /
ID of new entry.

4.1.9 Method: `nCreateNewTestCase`

Create bulk of test case entries: new test cases are buffered and inserted as bulk.

Once `_NUM_BUFFERD_ELEMENTS_FOR_EXECUTEMANY` is reached, the creation query is executed.

Arguments:

- `tbl.case.name`
/ *Condition*: required / *Type*: str /
Test case name.
- `tbl.case.issue`
/ *Condition*: required / *Type*: str /
Test case issue ID.
- `tbl.case.tcid`
/ *Condition*: required / *Type*: str /
Test case ID (used for testmanagement tool).

- `.tbl_case_fid`
/ *Condition*: required / *Type*: str /
Test case requirement (function) ID.
- `.tbl_case_testnumber`
/ *Condition*: required / *Type*: int /
Order of test case in file.
- `.tbl_case_repeatcount`
/ *Condition*: required / *Type*: int /
Test case repetition count.
- `.tbl_case_component`
/ *Condition*: required / *Type*: str /
Component which test case is belong to.
- `.tbl_case_time_start`
/ *Condition*: required / *Type*: str /
Test case start time as format %Y-%m-%d %H:%M:%S.
- `.tbl_case_result_main`
/ *Condition*: required / *Type*: str /
Test case main result.
- `.tbl_case_result_state`
/ *Condition*: required / *Type*: str /
Test case completion state.
- `.tbl_case_result_return`
/ *Condition*: required / *Type*: int /
Test case result code (as integer).
- `.tbl_case_counter_resets`
/ *Condition*: required / *Type*: int /
Counter of target reset within test case execution.
- `.tbl_case_lastlog`
/ *Condition*: required / *Type*: str /
Traceback information when test case is failed.
- `.tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result for linking to file in `tbl_result` table.
- `.tbl_file_id`
/ *Condition*: required / *Type*: int /
Test file ID for linking to file in `tbl_file` table.

Returns:*(no returns)*

4.1.10 Method: vCreateTags

Create tag entries.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `_tbl_usr_result_tags`
/ *Condition*: required / *Type*: str /
User tags information.

Returns:

(no returns)

4.1.11 Method: vSetCategory

Create category entry.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `tbl_result_category_main`
/ *Condition*: required / *Type*: str /
Category information.

Returns:

(no returns)

4.1.12 Method: vUpdateStartTimeEndTime

Create start-end time entry.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `_tbl_result_time_start`
/ *Condition*: required / *Type*: str /
Result start time as format %Y-%m-%d %H:%M:%S.
- `_tbl_result_time_end`
/ *Condition*: required / *Type*: str /
Result end time as format %Y-%m-%d %H:%M:%S.

Returns:

(no returns)

4.1.13 Method: arGetCategories

Get existing categories.

Arguments:

(no arguments)

Returns:

- arCategories
/ *Type*: list /
List of existing categories.

4.1.14 Method: vCreateAbortReason

Create abort reason entry.

Arguments:

- tbl_test_result_id
/ *Condition*: required / *Type*: str /
UUID of test result.
- tbl_abort_reason
/ *Condition*: required / *Type*: str /
Abort reason.
- tbl_abort_message
/ *Condition*: required / *Type*: str /
Detail message of abort.

Returns:

(no returns)

4.1.15 Method: vCreateReanimation

Create reanimation entry.

Arguments:

- tbl_test_result_id
/ *Condition*: required / *Type*: str /
UUID of test result.
- tbl_num_of_reanimation
/ *Condition*: required / *Type*: int /
Counter of target reanimation during execution.

Returns:

(no returns)

4.1.16 Method: vCreateCCRdata

Create CCR data per test case.

Arguments:

- tbl_test_case_id
/ *Condition*: required / *Type*: int /
test case ID.

- `lCCRdata`
/ *Condition*: required / *Type*: list /
list of CCR data.

Returns:*(no returns)***4.1.17 Method: vFinishTestResult**

Finish upload:

- First do bulk insert of rest of test cases if buffer is not empty.
- Then set state to "new report".

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:*(no returns)***4.1.18 Method: vUpdateEvtbls**Call `update_evtbls` stored procedure.**Arguments:***(no arguments)***Returns:***(no returns)***4.1.19 Method: vUpdateEvtbl**Call `update_evtbl` stored procedure to update provided `test_result_id`.**Arguments:**

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:*(no returns)***4.1.20 Method: vEnableForeignKeyCheck**Switch `foreign_key_checks` flag.**Arguments:**

- `enable`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, enable foreign key constraint.

Returns:*(no returns)*

4.1.21 Method: sGetLatestFileID

Get latest file ID from `tbl_file` table.

Arguments:

- `tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

- `tbl_file_id`
/ *Type*: int /
File ID.

4.1.22 Method: vUpdateFileEndTime

Update test file end time.

Arguments:

- `tbl_file_id`
/ *Condition*: required / *Type*: int /
File ID to be updated.
- `tbl_file_time_end`
/ *Condition*: required / *Type*: str /
File end time as format `%Y-%m-%d %H:%M:%S`.

Returns:

(no returns)

4.1.23 Method: vUpdateResultEndTime

Update test result end time.

Arguments:

- `tbl_test_result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be updated.
- `tbl_result_time_end`
/ *Condition*: required / *Type*: str /
Result end time as format `%Y-%m-%d %H:%M:%S`.

Returns:

(no returns)

4.1.24 Method: bExistingResultID

Verify the given test result UUID is existing in `tbl_result` table or not.

Arguments:

- `tbl_test_result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be verified.

Returns:

- `bExisting`
/ *Type*: bool /
True if test result UUID is already existing.

4.1.25 Method: `arGetProjectVersionSWByID`

Get the project and version_sw information of given `test_result_id`

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be get the information.

Returns:

- / *Type*: tuple /
None if test result UUID is not existing, else the tuple which contains project and version_sw: (project, variant) is returned.

Chapter 5

rest_api_db_access.py

5.1 Class: RestApiDBAccess

Imported by:

```
from TestResultDBAccess.DBAccess.rest_api_db_access import RestApiDBAccess
```

RestApiDBAccess class provide methods to interact with TestResultWebApp's REST APIs.

This class implements the **DBAccessInterface** and extends it. It includes methods for connecting to the database, handling API requests, creating, updating, and calling stored procedures in the database via RESTful API calls.

5.1.1 Method: get_certs_file

Retrieves SSL certificates and saves them to a temporary file for request's verification.

Returns:

/ Type: str /

The path to the temporary file containing SSL certificates.

5.1.2 Method: encrypt_password

Encrypts a password using a public key.

Arguments:

- password

/ Type: str /

The password to encrypt.

- pubkey

/ Type: str /

The public key used for encryption.

Returns:

/ Type: str /

The encrypted password.

5.1.3 Method: connect

Connects to the database via REST API using the provided credentials.

Arguments:

- `host`
/ *Condition*: required / *Type*: str /
URL which is hosted the TestResultWebApp's REST APIs.
- `user`
/ *Condition*: required / *Type*: str /
User name for database authentication.
- `passwd`
/ *Condition*: required / *Type*: str /
User's password for database authentication.
- `database`
/ *Condition*: required / *Type*: str /
Database name.

Returns:

(no returns)

5.1.4 Method: disconnect

Disconnect from TestResultWebApp's database.

Arguments:

(no arguments)

Returns:

(no returns)

5.1.5 Method: arGetCategories

Get existing categories.

Arguments:

(no arguments)

Returns:

- `arCategories`
/ *Type*: list /
List of existing categories.

5.1.6 Method: bExistingResultID

Verify the given test result UUID is existing or not.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be verified.

Returns:

/ *Type*: bool /
True if given `result_id` is already existing.

5.1.7 Method: sGetLatestFileID

Get latest file ID of all result or given `result_id`.

Arguments:

- `result_id`
/ *Condition*: optional / *Type*: str /
Test result ID. If used, the latest file id of given `result_id` is returned.

Returns:

/ *Type*: int /
Latest file ID.

5.1.8 Method: arGetProjectVersionSWByID

Get the project and version_sw information of given `result_id`

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be get the information.

Returns:

- / *Type*: tuple /
None if test result UUID is not existing, else the tuple which contains project and version_sw: (project, variant) is returned.

5.1.9 Method: sCreateNewTestResult

Creates a new test result.

Arguments:

- `project`
/ *Condition*: required / *Type*: str /
Project information.
- `variant`
/ *Condition*: required / *Type*: str /
Variant information.
- `branch`
/ *Condition*: required / *Type*: str /
Branch information.
- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `result_interpretation`
/ *Condition*: required / *Type*: str /
Result interpretation.

- `result_start_time`
/ *Condition*: required / *Type*: str /
Test result start time as format %Y-%m-%d %H:%M:%S.
- `result_end_time`
/ *Condition*: required / *Type*: str /
Test result end time as format %Y-%m-%d %H:%M:%S.
- `result_version_sw_target`
/ *Condition*: required / *Type*: str /
Software version information.
- `result_version_sw_test`
/ *Condition*: required / *Type*: str /
Test version information.
- `result_version_hw`
/ *Condition*: required / *Type*: str /
Hardware version information.
- `result_build_url`
/ *Condition*: required / *Type*: str /
Link to the execution of test result (Jenkins, Gitlab CI/CD, ...).
- `result_report_qualitygate`
/ *Condition*: required / *Type*: str /
Quality gate information for reporting.

Returns:

- `result_id`
/ *Type*: str /
`test_result_id` of new test result.

5.1.10 Method: nCreateNewFile

Create new result file.

Arguments:

- `file_name`
/ *Condition*: required / *Type*: str /
File name information.
- `file_tester_account`
/ *Condition*: required / *Type*: str /
Tester account information.
- `file_tester_machine`
/ *Condition*: required / *Type*: str /
Test machine information.
- `file_time_start`
/ *Condition*: required / *Type*: str /
Test file start time as format %Y-%m-%d %H:%M:%S.

- `file_time_end`
/ *Condition*: required / *Type*: str /
Test file end time as format %Y-%m-%d %H:%M:%S.
- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result to which this result file belongs.
- `file_origin`
/ *Condition*: required / *Type*: str /
Origin (test framework) of test file.”

Returns:

/ *Type*: int /
ID of new entry.

5.1.11 Method: vCreateNewHeader

Create a new result file header.

Arguments:

- `file_id`
/ *Condition*: required / *Type*: int /
File ID information.
- `testtoolconfiguration.testtoolname`
/ *Condition*: required / *Type*: str /
Test tool name.
- `testtoolconfiguration.testtoolversionstring`
/ *Condition*: required / *Type*: str /
Test tool version.
- `testtoolconfiguration.projectname`
/ *Condition*: required / *Type*: str /
Project name.
- `testtoolconfiguration.logfileencoding`
/ *Condition*: required / *Type*: str /
Encoding of logfile.
- `testtoolconfiguration.pythonversion`
/ *Condition*: required / *Type*: str /
Python version info.
- `testtoolconfiguration.testfile`
/ *Condition*: required / *Type*: str /
Test file name.
- `testtoolconfiguration.logfilepath`
/ *Condition*: required / *Type*: str /
Path to log file.
- `testtoolconfiguration.logfilemode`
/ *Condition*: required / *Type*: str /
Mode of log file.

- `testtoolconfiguration.ctrlfilepath`
/ *Condition*: required / *Type*: str /
Path to control file.
- `testtoolconfiguration.configfile`
/ *Condition*: required / *Type*: str /
Path to configuration file.
- `testtoolconfiguration.confname`
/ *Condition*: required / *Type*: str /
Configuration name.
- `testfileheader.author`
/ *Condition*: required / *Type*: str /
File author.
- `testfileheader.project`
/ *Condition*: required / *Type*: str /
Project information.
- `testfileheader.testfiledate`
/ *Condition*: required / *Type*: str /
File creation date.
- `testfileheader.version.major`
/ *Condition*: required / *Type*: str /
File major version.
- `testfileheader.version.minor`
/ *Condition*: required / *Type*: str /
File minor version.
- `testfileheader.version.patch`
/ *Condition*: required / *Type*: str /
File patch version.
- `testfileheader.keyword`
/ *Condition*: required / *Type*: str /
File keyword.
- `testfileheader.shortdescription`
/ *Condition*: required / *Type*: str /
File short description.
- `testexecution.useraccount`
/ *Condition*: required / *Type*: str /
Tester account who run the execution.
- `testexecution.computername`
/ *Condition*: required / *Type*: str /
Machine name which is executed on.
- `testrequirements.documentmanagement`
/ *Condition*: required / *Type*: str /
Requirement management information.

- `testrequirements.testenvironment`
/ *Condition*: required / *Type*: str /
Requirement environment information.
- `testbenchconfig.name`
/ *Condition*: required / *Type*: str /
Test bench configuration name.
- `testbenchconfig.data`
/ *Condition*: required / *Type*: str /
Test bench configuration data.
- `preprocessor.filter`
/ *Condition*: required / *Type*: str /
Preprocessor filter information.
- `preprocessor.parameters`
/ *Condition*: required / *Type*: str /
Preprocessor parameters definition.

Returns:

(no returns)

5.1.12 Method: `nCreateNewSingleTestCase`

Create single test case.

Arguments:

- `case_name`
/ *Condition*: required / *Type*: str /
Test case name.
- `case_issue`
/ *Condition*: required / *Type*: str /
Test case issue ID.
- `case_tcid`
/ *Condition*: required / *Type*: str /
Test case ID (used for testmanagement tool).
- `case_fid`
/ *Condition*: required / *Type*: str /
Test case requirement (function) ID.
- `case_testnumber`
/ *Condition*: required / *Type*: int /
Order of test case in file.
- `case_repeatcount`
/ *Condition*: required / *Type*: int /
Test case repeatition count.
- `case_component`
/ *Condition*: required / *Type*: str /
Component which test case is belong to.

- `case_time_start`
/ *Condition*: required / *Type*: str /
Test case start time as format %Y-%m-%d %H:%M:%S.
- `case_result_main`
/ *Condition*: required / *Type*: str /
Test case main result.
- `case_result_state`
/ *Condition*: required / *Type*: str /
Test case completion state.
- `case_result_return`
/ *Condition*: required / *Type*: int /
Test case result code (as integer).
- `case_counter_resets`
/ *Condition*: required / *Type*: int /
Counter of target reset within test case execution.
- `case_lastlog`
/ *Condition*: required / *Type*: str /
Traceback information when test case is failed.
- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result to which this test case belongs.
- `file_id`
/ *Condition*: required / *Type*: int /
ID of result file to which this test case belongs.

Returns:

/ *Type*: int /
ID of new entry.

5.1.13 Method: nCreateNewTestCase

Alias for `nCreateNewSingleTestCase`, used in older import tools to import a bulk of test cases at once.

5.1.14 Method: vCreateAbortReason

Create abort reason entry.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `abort_reason`
/ *Condition*: required / *Type*: str /
Abort reason.
- `abort_message`
/ *Condition*: required / *Type*: str /
Detail message of abort.

Returns:

(no returns)

5.1.15 Method: vCreateCCRdata

Create CCR data per test case.

Arguments:

- `tbl_test_case_id`
/ *Condition*: required / *Type*: int /
test case ID.
- `lCCRdata`
/ *Condition*: required / *Type*: list /
list of CCR data.

Returns:

(no returns)

5.1.16 Method: vCreateTags

Create tag entries.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `tags`
/ *Condition*: required / *Type*: str /
User tags information.

Returns:

(no returns)

5.1.17 Method: vCreateReanimation

Create reanimation entry.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `num_of_reanimation`
/ *Condition*: required / *Type*: int /
Counter of target reanimation during execution.

Returns:

(no returns)

5.1.18 Method: vSetCategory

Create category entry.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `category_main`
/ *Condition*: required / *Type*: str /
Category information.

Returns:

(no returns)

5.1.19 Method: vUpdateFileEndTime

Update test file end time.

Arguments:

- `file_id`
/ *Condition*: required / *Type*: int /
File ID to be updated.
- `time_end`
/ *Condition*: required / *Type*: str /
File end time as format %Y-%m-%d %H:%M:%S.

Returns:

(no returns)

5.1.20 Method: vUpdateResultEndTime

Update test result end time.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be updated.
- `time_end`
/ *Condition*: required / *Type*: str /
Result end time as format %Y-%m-%d %H:%M:%S.

Returns:

(no returns)

5.1.21 Method: vFinishTestResult

Update state of given test result to "new report".

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

(no returns)

5.1.22 Method: vUpdateEvtbl

Call `update_evtbl` stored procedure to update given `result_id`.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

(no returns)

5.1.23 Method: vUpdateEvtbls

Call `update_evtbls` stored procedure.

Arguments:

(no arguments)

Returns:

(no returns)

Chapter 6

DBAccessFactory.py

6.1 Class: DBAccessFactory

Imported by:

```
from TestResultDBAccess.DBAccessFactory import DBAccessFactory
```

6.1.1 Method: create

Chapter 7

Appendix

About this package:

Table 7.1: Package setup

Setup parameter	Value
Name	TestResultDBAccess
Version	0.1.5
Date	10.04.2025
Description	Interfaces to access TestResultWebApp database
Package URL	python-testresultdbaccess
Author	Tran Duy Ngoan
Email	Ngoan.TranDuy@vn.bosch.com
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 8

History

0.1.0	22.04.2024
<i>Initial version</i>	
0.1.1	15.05.2024
<i>Fixed issues of workflow to publish package to pypi</i>	
0.1.2	17.05.2024
<i>Added requirements_linux.sh file, which helps to install Linux dependency packages</i>	
0.1.3	17.05.2024
<i>Corrected link to package on pypi</i>	
0.1.4	01.08.2024
<i>- Fixed typos and renamed files properly - Fixed wam issue with test account due to ssl verification</i>	
0.1.5	10.04.2025
<i>Add step to get cookies by using webdriver in case of failure kerberos request</i>	

TestResultDBAccess.pdf*Created at 10.04.2025 - 14:42:30**by GenPackageDoc v. 0.16.0*
